

Architektura i implementacja zaawansowanych narzędzi kadrowania i prostowania

Model architektoniczny

Najsolidniejszy fundament dla takiego edytora to architektura niedestrukcyjna, w której stan edycji jest oddzielony od rastra źródłowego. W praktyce oznacza to trzy warstwy: model stanu, renderer podglądu oraz pipeline eksportu i analizy. Na WWW taki renderer można oprzeć o `canvas`, który jest przeznaczony także do manipulacji fotografią; przy złożonych scenach dobrze działa układ wielu warstw `canvas`, gdzie osobno żyje tło obrazu, osobno nakładki i osobno interaktywne uchwyty. Jeśli podgląd lub analiza robią się ciężkie, `OffscreenCanvas` pozwala przenieść renderowanie do workera, tak aby główny wątek interfejsu nie tracił responsywności. ¹

W takim modelu nie warto zapisywać kadru w pikselach ekranu. Lepszą reprezentacją jest stan znormalizowany względem oryginalnego obrazu, np. $x, y, w, h \in [0, 1]$, plus niezależne parametry transformacji, takie jak obrót, zoom, przesunięcie i aktywna nakładka. To nie jest tylko teoria: popularne komponenty cropujące jawnie rozróżniają współrzędne pikselowe i procentowe, a dokumentacja podkreśla, że procentowe kadry są odporne na zmianę rozmiaru obrazu lub viewportu. Ten sam kierunek widać w dojrzałych edytorach fotograficznych, gdzie przycięcie jest operacją odwracalną i nie niszczy pliku źródłowego. Przeliczenie do eksportu jest wtedy trywialne: $x_s = x_n W_s$, $y_s = y_n H_s$, $w_s = w_n W_s$, $h_s = h_n H_s$, gdzie W_s, H_s są rozmiarami oryginału. ²

Trzeba też świadomie rozdzielić cztery układy współrzędnych: piksele CSS, piksele bufora renderera, piksele obrazu źródłowego i współrzędne znormalizowane. To rozdzielenie jest kluczowe dla ostrości siatek i uchwytów na ekranach HiDPI. Dokumentacja `devicePixelRatio` wprost wskazuje, że `canvas` może być rozmyty na ekranach Retina, jeśli rozmiar bufora nie zostanie przeskalowany względem pikseli CSS. W praktyce renderer powinien ustawiać `canvas.width = cssWidth * dpr`, `canvas.height = cssHeight * dpr`, a następnie skalować kontekst o `dpr`. ³

Nakładki pomocnicze

Nakładki kompozycyjne najlepiej potraktować jako czysto wektorową warstwę nad podglądem: rysowaną w osobnym `canvas`, w SVG albo jako pass shaderowy w WebGL. Po stronie Canvas API `Path2D` jest szczególnie użyteczny, bo pozwala przechowywać i wielokrotnie odrysowywać złożone ścieżki, kopiować istniejące ścieżki oraz budować je bezpośrednio ze stringów SVG path. To dobrze pasuje do siatek i spiral, które nie powinny dotykać samego bufora obrazu. Dodatkowo wzorzec „indeksu bieżącej nakładki” jest bardzo sensowny z UX-owego punktu widzenia; duże edytory stosują dokładnie taki model interakcji, w którym klawisz `0` przełącza kolejne typy siatki. ⁴

Siatka trójpodziału jest najprostsza: pionowe linie trafiają w $x = \frac{W}{3}$ i $x = \frac{2W}{3}$, a poziome w $y = \frac{H}{3}$ i $y = \frac{2H}{3}$. Nakładka złotego podziału powinna bazować na $\phi = \frac{1+\sqrt{5}}{2} \approx 1,618$, czyli dodatnim pierwiastku równania $x^2 - x - 1 = 0$. Dla prostokąta o szerokości W i wysokości H sensowne pozycje linii to $x = \frac{W}{\phi}$, $x = W - \frac{W}{\phi}$, $y = \frac{H}{\phi}$, $y = H - \frac{H}{\phi}$. To nie jest tylko matematyczna abstrakcja: dokumentacja narzędzi fotograficznych opisuje „Golden” jako siatkę Phi opartą na złotej proporcji. Z

kolei spirala złota wynika z kolejnych punktów dzielących złoty prostokąt na kwadraty, które leżą na spirali logarytmicznej. ⁵

Nakładki diagonalne i trójkątne można potraktować jako warianty tej samej geometrii projekcyjnej. Dla diagonalni podstawowej bierzemy odcinek AB z jednego rogu do przeciwległego rogu. Dla pozostałych rogów C i D wyliczamy rzuty prostopadłe na prostą AB , a następnie rysujemy odcinki $C \rightarrow P_C$ oraz $D \rightarrow P_D$. Jeżeli $A = (0, 0)$, $B = (W, H)$, a $d = \frac{B-A}{\|B-A\|}$, to stopa prostopadłej z punktu C wynosi $P_C = A + \langle C - A, d \rangle d$. W praktyce warto trzymać oba kierunki diagonalne jako dwa osobne tryby lub jako wariant orientacji, bo to pozwala zachować spójny mechanizm rotacji nakładki bez przebudowy logiki rysowania.

Sama obsługa klawisza może być banalna, o ile jest dobrze odseparowana od renderera obrazu:

```
const overlayModes = ['thirds', 'phi', 'spiral', 'diagonalA', 'diagonalB', 'triangleA', 'triangleB'];

function onKeyDown(e: KeyboardEvent) {
  if (e.key.toLowerCase() === 'o') {
    state.overlayIndex = (state.overlayIndex + 1) % overlayModes.length;
    invalidateOverlayOnly();
  }
}
```

Kluczowe jest tutaj `invalidateOverlayOnly()`: zmienia się wyłącznie warstwa wektorowa, nie raster zdjęcia.

Ramka kadrowania i współczynnik proporcji

Do uchwytów kadrowania lepiej używać Pointer Events niż natywnego HTML Drag and Drop. Specyfikacja Pointer Events dostarcza jednolity model dla myszy, dotyku i pióra, a pointer capture pozwala utrzymać strumień zdarzeń nawet wtedy, gdy kursor wyjedzie poza sam uchwyt. Równie ważne jest ustawienie `touch-action` na powierzchni edytora, bo inaczej przeglądarka może przejąć gesty przewijania lub pinch-zoom i zakończyć interakcję `pointercancel`. Trzeba też pamiętać, że `pointermove` może odpalać się z bardzo dużą częstotliwością; dlatego dobrym wzorcem jest aktualizowanie stanu na bieżąco, ale renderowanie ograniczać do `requestAnimationFrame`, a tam, gdzie liczy się precyzja, można korzystać z `getCoalescedEvents()`. ⁶

Stan prostokąta kadrowania warto przechowywać jako znormalizowane x, y, w, h i zawsze liczyć nową geometrię z „migawki” stanu z momentu `pointerdown`, a nie z poprzedniego `pointermove`. To eliminuje dryf numeryczny i błędy akumulacji. W praktyce biblioteki cropujące pokazują dokładnie ten kierunek: zwracają równoległe wymiary w pikselach i w procentach, przy czym procenty są odporne na zmianę rozmiaru medium. Jednocześnie dokumentacja dojrzałych edytorów fotograficznych potwierdza zachowanie kłódki aspektu: przy zamkniętej blokadzie szerokość i wysokość zmieniają się razem, a przy otwartej mogą być korygowane niezależnie. ²

Algorytm blokady proporcji najlepiej zbudować wokół aktywnego uchwytu i nieruchomego punktu kotwiczącego. Jeśli użytkownik ciągnie np. prawą krawędź, podstawową niewiadomą jest nowa szerokość w' , a wysokość wyliczamy jako $h' = \frac{w'}{r}$, gdzie r jest zablokowanym ratio, np. $16/9$. Jeśli

aktywny jest uchwyt narożny, trzeba rozstrzygnąć, która oś jest „dominująca” dla danego ruchu. Dobrze działa heurystyka: w znormalizowanej przestrzeni obrazu porównaj $|\Delta x|$ z $|\Delta y| \cdot r$; większa wartość wybiera oś sterującą, a druga jest rekalkulowana. Dzięki temu użytkownik nie ma wrażenia, że prostokąt „walczy” z kursorem. Dokumentacja jednego z popularnych komponentów wprost sygnalizuje też, że aspekty w procentach bywają zdradliwe, dlatego taki krok korekcyjny powinien być jawnie zaimplementowany, a nie pozostawiony przypadkowi. ⁷

Najważniejszy detal produkcyjny to edge constraints. Po każdej propozycji nowego prostokąta trzeba przeprowadzić testy graniczne względem dostępnego obszaru obrazu, a po włączeniu rotacji względem ważnego obszaru po transformacji, nie tylko względem prostokąta osiowego. Dla nierotowanego przypadku warunek jest prosty: $0 \leq x \leq 1, 0 \leq y \leq 1, x + w \leq 1, y + h \leq 1$. Jeżeli po korekcie aspektu otrzymamy $x + w' > 1$, to redukujemy w' do $1 - x$ i dopiero z tego ponownie liczymy h' ; analogicznie dla dolnej krawędzi. W dobrze zaprojektowanej implementacji to właśnie stan jest „obcinany”, a nie sam kursor. Użytkownik powinien widzieć, że uchwyt zatrzymał się na prawidłowej granicy zdjęcia, a nie że interfejs się zaciął.

Przyciemnienie obszaru poza kadrem też warto robić na warstwie UI, a nie przez kopiowanie obrazu. `clip()` pozwala zamienić ścieżkę w region przycinający, a `Path2D` plus reguła `evenodd` są wygodne do modelu „pełen ekran minus crop rect”. Trzeba tylko pamiętać, że regionu clip nie da się „cofnąć” inaczej niż przez `save()` i `restore()`, więc warstwa overlay powinna wchodzić i wychodzić z tego stanu w sposób kontrolowany. ⁸

Prostowanie ręczne

Narzędzie poziomiczy jest matematycznie proste, ale implementacyjnie zdradliwe. Jeśli użytkownik wskazuje dwa punkty (x_1, y_1) i (x_2, y_2) , to kąt linii względem dodatniej osi x wyznacza $\theta = \text{atan2}(y_2 - y_1, x_2 - x_1)$. Dokumentacja `Math.atan2()` definiuje ten wynik jako kąt w radianach między osią x a promieniem od $(0, 0)$ do (x, y) . Jeżeli wskazana linia ma stać się nowym horyzontem, korekta obrotu wynosi zazwyczaj $-\theta$. W praktyce warto jeszcze odrzucać bardzo krótkie odcinki, bo dla nich najmniejszy błąd dotyku lub myszy daje nieproporcjonalnie duży błąd kąta. ⁹

Po stronie renderera masz dwa równoważne światy. W Canvas 2D można korzystać z `setTransform()`, które resetuje macierz do identyczności i dopiero potem stosuje nową transformację; to wygodne, bo eliminuje przypadkowe narastanie transformacji między klatkami. W silnikach graficznych i backendach CV odpowiednikiem jest macierz afiniczna obrotu. OpenCV podaje ją jawnie jako macierz 2×3 , w której $\alpha = s \cos \theta, \beta = s \sin \theta$, a środek obrotu jest mapowany na samego siebie. Następnie `warpAffine()` nakłada tę transformację na obraz. To ważne, bo oznacza, że straighten tool nie jest „obracaniem DOM-u”, tylko pełnoprawną transformacją geometrii obrazu. ¹⁰

Kluczowy detal jakościowy to sposób próbkowania pikseli. Dokumentacja OpenCV podkreśla, że transformacje geometryczne są realizowane odwrotnie, od obrazu wynikowego do źródła, aby uniknąć artefaktów próbkowania; przy współrzędnych niecałkowitych pojawia się interpolacja, a biblioteka udostępnia warianty od nearest neighbor przez bilinear i bicubic po Lanczos. To jest bardzo ważne dla straighten tool: podgląd może spokojnie używać bilinearu, ale eksport nie powinien bezrefleksyjnie polegać na domyślnym wygładzaniu przeglądarki. W Canvas 2D istnieje `imageSmoothingQuality`, lecz dokumentacja zastrzega, że ten mechanizm nie jest Baseline i ma domyślną wartość „low”. Produkcyjnie lepiej więc traktować Canvas jako engine podglądu, a finalny eksport oprzeć o kontrolowany resampler. ¹¹

Jeśli podgląd ma działać bardzo płynnie przy dużych plikach lub wielu jednoczesnych transformacjach, warto rozważyć renderer GPU. W WebGL kontekst działa na `canvas`, macierze trafiają do shaderów przez `uniformMatrix*fv()`, a same shadery wykonują się bezpośrednio w potoku graficznym GPU. To daje naturalny model dla rotacji, skali i translacji jako jednej macierzy transformacji, przy czym CPU zostaje odciążony od przeliczania każdego piksela w JavaScriptcie. ¹²

Po obrocie obraz przestaje pokrywać pełny prostokąt osiowy podglądu; geometrycznie staje się obróconym czworokątem. Dlatego edge constraints dla crop boxa nie mogą już sprawdzać wyłącznie warunków typu $x + w \leq 1$. Albo testujesz zawieranie w wielokącie obróconego obrazu, albo – co zwykle prostsze – wykonujesz wszystkie testy w przestrzeni źródłowej po transformacji odwrotnej. To jest spójne z tym, jak działają dojrzałe biblioteki CV: najpierw definiuje się mapowanie, potem próbuje się źródło do docelowych pikseli. ¹³

Auto prostowanie

Klasyczny pipeline auto-straighten nadal ma bardzo mocne uzasadnienie architektoniczne. Najpierw bierzesz obraz proxy, opcjonalnie go odsumiasz, liczysz gradienty Sobela albo uruchamiasz Canny'ego, a na obrazie krawędzi uruchamiasz HoughLines lub HoughLinesP. Dokumentacja OpenCV wyraźnie pokazuje, że Sobel służy do wyznaczania pochodnych i gradientów, Canny jest wieloetapowym detektorem krawędzi, a Hough do wykrywania linii prostych powinien dostawać jako wejście obraz po preprocessingu krawędziowym. To dokładnie odpowiada potrzebie automatycznego prostowania opartego o horyzont, krawędzie budynków i inne dominujące linie strukturalne. ¹⁴

Sama detekcja linii to dopiero połowa pracy. Potem trzeba wyliczyć kąt korekty w sposób odporny na śmieci. Dla segmentu o końcach (x_1, y_1) , (x_2, y_2) obliczasz $\theta_i = \text{atan2}(y_2 - y_1, x_2 - x_1)$, filtrujesz segmenty „prawie poziome” i „prawie pionowe”, ważysz je np. długością segmentu i siłą odpowiedzi, a następnie sprowadzasz do odchylenia od najbliższej osi. Bardzo dobrze działa średnia kołowa w podwójnym kącie:

$$\hat{\delta} = \frac{1}{2} \text{atan2} \left(\sum_i w_i \sin 2\delta_i, \sum_i w_i \cos 2\delta_i \right),$$

gdzie δ_i jest odchyleniem segmentu od 0° lub 90° . Ta postać dobrze obchodzi symetrię linii, bo kierunki θ i $\theta + \pi$ opisują tę samą prostą. To jest właśnie ten moment, w którym „auto” przestaje być prostym Hough + average, a staje się narzędziem jakościowym.

Podejście głębokie staje się dużo lepsze wtedy, gdy fotografia nie ma czystego, jawnego horyzontu albo gdy klasyczne metody mylą strukturę sceny z przeszkodami. Praca *Horizon Lines in the Wild* wprost pokazuje, że CNN może estymować linię horyzontu bez narzucania explicite geometrycznych ograniczeń i bez potrzeby obecności klasycznych wskazówek, takich jak regularne tekstury czy punkty zbiegu. Jeszcze ciekawszy dla Twojego przypadku jest nurt Deep Hough Transform: jedna praca przenosi reprezentacje do domeny parametrycznej linii, agregując cechy wzdłuż linii kandydackich i redukując post-processing do szukania punktów w przestrzeni parametrów; druga dodaje do sieci trenowalny blok Hougha jako globalny prior geometrii linii i raportuje lepszą efektywność danych. To jest dokładnie ten typ hybrydy, który dobrze pasuje do auto-straighten dla „trudnych” zdjęć. ¹⁵

Najrozsądniejszy kompromis produkcyjny to więc nie wybór „albo klasyka, albo AI”, tylko strategia kaskadowa. Najpierw szybki pipeline klasyczny na proxy, bo jest przewidywalny, tani obliczeniowo i łatwy do debugowania. Jeżeli wynik ma niską pewność — na przykład ma mało inlierów, rozjechane dominanty poziome i pionowe albo duży rozrzut estymat — wtedy uruchamiasz model semantyczny.

Taki układ daje dobrze skalowalny koszt obliczeń i eliminuje najczęstsze porażki czystego Hougha na scenach zasłoniętych przez góry, drzewa, statki albo nieregularną architekturę. ¹⁶

Rekomendowana implementacja produkcyjna

W praktyce najlepsza implementacja „super zaawansowanego” zestawu narzędzi wygląda tak: interfejs wejściowy działa na Pointer Events, warstwa wizualna używa osobnych warstw `canvas` dla obrazu i overlayu, podgląd rotacji i zoomu działa na lekkim rendererze 2D albo WebGL, analiza auto-straighten biegnie na obrazie proxy w workerze, a eksport zawsze wykonuje pełnoprawne przeliczenie na obrazie źródłowym. Taka architektura łączy responsywność warstwowego `canvas`, zalety workera i OffscreenCanvas oraz niedestrukcyjny model stanu z procentowym crop boxem. ¹⁷

Dobrym punktem startowym jest taki model stanu:

```
type OverlayMode =
  | 'thirds'
  | 'phi'
  | 'spiral'
  | 'diagonalA'
  | 'diagonalB'
  | 'triangleA'
  | 'triangleB'
  | 'none';

interface CropRectNorm {
  x: number; // 0..1
  y: number; // 0..1
  w: number; // 0..1
  h: number; // 0..1
}

interface EditorState {
  crop: CropRectNorm;
  rotationRad: number;
  zoom: number;
  panX: number;
  panY: number;
  aspectLock: number | null; // np. 16/9, 1, 4/5
  overlayMode: OverlayMode;
  overlayOrientation: number;
  autoStraightenConfidence: number;
}
```

Najważniejsze jest to, że ten model nie zależy ani od Canvas 2D, ani od WebGL, ani od konkretnej biblioteki CV. Dzięki temu możesz bez bólu wymienić renderer lub backend analityczny.

Jeżeli miałbym streścić rekomendację w jednym zdaniu, byłaby taka: **nakładki jako czysta geometria wektorowa, kadrowanie jako znormalizowany stan niedestrukcyjny, prostowanie ręczne jako jawna transformacja afiniczna sterowana `atan2`, a auto-prostowanie jako kaskada klasycznego**

wykrywania linii z opcjonalnym modułem Deep Hough / horizon-estimation dla scen niejednoznacznych. Taki układ skaluje się od prostego edytora webowego do pełnego narzędzia klasy desktop bez konieczności zmiany podstawowego modelu danych i bez mieszania logiki UI z logiką przetwarzania obrazu. ¹⁸

- ¹ ¹⁷ https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Optimizing_canvas
https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Optimizing_canvas
- ² ⁷ <https://github.com/dominictobias/react-image-crop>
<https://github.com/dominictobias/react-image-crop>
- ³ <https://developer.mozilla.org/en-US/docs/Web/API/Window/devicePixelRatio>
<https://developer.mozilla.org/en-US/docs/Web/API/Window/devicePixelRatio>
- ⁴ ¹⁸ <https://developer.mozilla.org/en-US/docs/Web/API/Path2D/Path2D>
<https://developer.mozilla.org/en-US/docs/Web/API/Path2D/Path2D>
- ⁵ Golden Ratio -- from Wolfram MathWorld
<https://mathworld.wolfram.com/GoldenRatio.html>
- ⁶ <https://www.w3.org/TR/pointerevents/>
<https://www.w3.org/TR/pointerevents/>
- ⁸ <https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/clip>
<https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/clip>
- ⁹ https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/atan2
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/atan2
- ¹⁰ <https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/setTransform>
<https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/setTransform>
- ¹¹ ¹³ https://docs.opencv.org/3.4/da/d54/group_imgproc_transform.html
https://docs.opencv.org/3.4/da/d54/group_imgproc_transform.html
- ¹² <https://developer.mozilla.org/en-US/docs/Web/API/WebGLRenderingContext>
<https://developer.mozilla.org/en-US/docs/Web/API/WebGLRenderingContext>
- ¹⁴ https://docs.opencv.org/3.4/d2/d2c/tutorial_sobel_derivatives.html
https://docs.opencv.org/3.4/d2/d2c/tutorial_sobel_derivatives.html
- ¹⁵ <https://arxiv.org/abs/1604.02129>
<https://arxiv.org/abs/1604.02129>
- ¹⁶ https://docs.opencv.org/4.x/d9/db0/tutorial_hough_lines.html
https://docs.opencv.org/4.x/d9/db0/tutorial_hough_lines.html